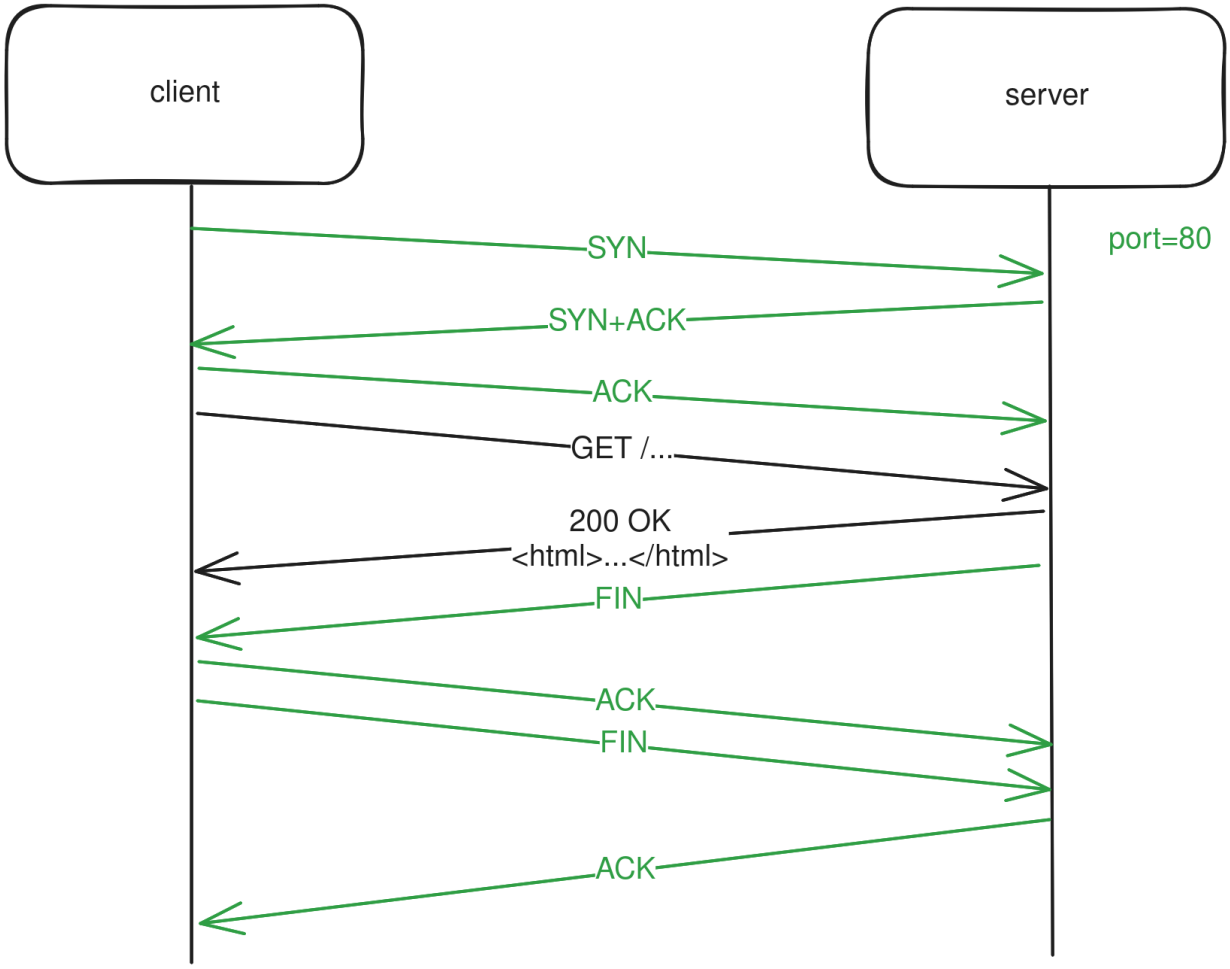
# INFOB315 - Télécommunications et réseaux
*Serveur HTTP en Python*

## 1 Serveur HTTP

Le but de ce TP est de créer un serveur HTTP en Python. La figure 1 reprend les échanges entre le client et le serveur (votre code Python) lors de la récupération d'une page HTML. Les échanges en lien avec le protocole TCP sont représentés au moyen de flèches vertes et les échanges utilisant le protocole HTTP avec des flèches noires.

Figure 1 – Échanges entre le client et le serveur



## 1.1 Compléments sur les sockets

Comme pour la partie client, votre serveur devra utiliser les *sockets* afin de permettre aux clients de se connecter. Le module `socket` (https://docs.python.org/3/library/socket.html) met à votre disposition la méthode `socket.bind()`. Cette méthode permet de signaler au système d'exploitation que votre programme (le serveur HTTP) utilise le port 80.

Contrairement au client, vous n'aurez pas besoin de la méthode `socket.connect()` puisque le serveur ne se connecte pas au client mais attend que les clients se connectent à lui. Afin d'attendre les clients, la méthode `socket.listen()` doit être utilisée.

## 1.2 Code Python

En Python, implémentez la fonction `run_server(hostname: str, port: int)` prenant en paramètres le nom d'hôte de votre serveur ainsi que le port sur lequel le serveur va écouter. Ce serveur web devra renvoyer une page web quand un client se connecte sur la page `/hello`. Il serait également intéressant de conserver des *logs* des connexions effectuées par les clients. Ces *logs* afficheraient ainsi l'adresse IP des clients, la ressource à laquelle ils veulent accéder et le code de retour qui leur est renvoyé.

```
Connection from 127.0.0.1:45240
GET /hello -> HTTP/1.1 200 OK
Connection from 127.0.0.1:45246
GET /favicon.ico -> HTTP/1.1 404 Not Found
```

## 1.3 Gestion des clients

Votre serveur est maintenant capable de gérer la requête d'un client et de lui renvoyer une réponse HTTP valide. Cependant, en pratique, un serveur devrait être capable de traiter les requêtes de plusieurs clients en même temps.

Afin de mieux comprendre les limitations du code actuel, créez une nouvelle route `/slow` sur votre serveur. Cette route renvoie également une page HTML mais effectue un *sleep* de 10 secondes afin de simuler une requête qui prendrait du temps à être traitée.

Accédez à la route `/slow` avec un navigateur. Normalement, la page devrait prend 10 secondes avant d'apparaître. Rafraîchissez la page `/slow` et, dans les 10 secondes, tentez d'accéder à la page `/hello`.

Que constatez-vous ? Comment pourriez-vous améliorer votre serveur HTTP ?